

BÀI 1: TỔNG QUAN VỀ KIỂM THỬ VÀ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

Mục tiêu:

Bài học cung cấp cho sinh viên kiến thức về vai trò và tầm quan trọng của kiểm thử phần mềm: vai trò của kiểm thử trong vòng đời phát triển phần mềm và ý nghĩa của nó trong việc nâng cao chất lượng, độ tin cậy, và tính bảo trì của phần mềm.

Qua đó sinh viên sẽ nắm vững các loại kiểm thử phần mềm: Bao gồm kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử hệ thống, và kiểm thử chấp nhận, cũng như hiểu rõ thời điểm và cách áp dụng từng loại kiểm thử.

Ngoài ra, sinh viên cần hiểu về mô hình CMMI (Capability Maturity Model Integration) là một khung tiêu chuẩn được phát triển để cải tiến quy trình quản lý và phát triển phần mềm trong lĩnh vực Đảm bảo Chất lượng Phần mềm.

1.1. Các định nghĩa cơ bản về kiểm thử phần mềm

Lỗi (Error): Lỗi là những vấn đề mà con người mắc phải trong quá trình phát triển các sản phẩm phần mềm. Trong thực tế, con người luôn có thể phạm lỗi. Khi lập trình viên phạm lỗi trong lập trình, ta gọi các lỗi đó là bug (con bọ). Lỗi có thể phát tán. Chẳng hạn, một lỗi về xác định yêu cầu có thể dẫn đến sai lầm về thiết kế và càng sai khi lập trình theo thiết kế này. Lỗi là nguyên nhân dẫn đến sai.

Sai (Fault): Sai là kết quả của lỗi, hay nói khác đi, lỗi sẽ dẫn đến sai. Cũng có thể nói sai là một biểu diễn của lỗi dưới dạng một biểu thức, chẳng hạn chương trình, văn bản, sơ đồ dòng dữ liệu, biểu đồ lớp,... Sai lầm có thể khó bị phát hiện. Khi nhà thiết kế mắc lỗi bỏ sót trong quá trình thiết kế, sai kết quả từ lỗi đó là thiếu mất cái gì đó mà lẽ ra cần phải có. Sai về nhiệm vụ xuất hiện khi vào sai thông tin, còn sai về bỏ quên xuất hiện khi không vào đủ thông tin. Loại sai thứ hai khó phát hiện và khó sửa hơn loại sai thứ nhất.

Thất bại (Failure): Thất bại xuất hiện khi một lỗi được thực thi. Có hai điều cần lưu ý ở đây. Một là thất bại chỉ xuất hiện dưới dạng có thể chạy được mà thông thường là mã nguồn. Hai là các thất bại chỉ liên kết với các lỗi về nhiệm vụ. Còn các thất bại tương ứng với các lỗi về bỏ quên thì xử lý thế nào? Những cái lỗi không bao giờ được tiến hành, hoặc không được tiến hành trong khoảng thời gian dài cần được xử lý thế nào? Virus Michaelangelo là một ví dụ về lỗi loại này. Nó chỉ được

tiến hành vào ngày sinh của Michaelangelo, tức ngày 6/3 mà thôi. Việc khảo sát có thể ngăn chặn nhiều thất bại bằng cách tìm ra các lỗi thuộc cả hai loại.

Sự cố (Incident): Khi thất bại xuất hiện, nó có thể hiển thị hoặc không, tức là rõ ràng hoặc không rõ ràng đối với người dùng hoặc người kiểm thử. Sự cố là triệu chứng liên kết với một thất bại và thể hiện cho người dùng hoặc người kiểm thử về sự xuất hiện của thất bại này.

Yêu cầu của khách hàng và đặc tả của phần mềm: Phần mềm được viết để thực hiện các nhu cầu của khách hàng. Các nhu cầu của khách hàng được thu thập, phân tích và khảo cứu và là cơ sở để quyết định chính xác các đặc trưng cần thiết mà sản phẩm phần mềm cần phải có. Dựa trên yêu cầu của khách hàng và các yêu cầu bắt buộc khác, đặc tả được xây dựng để mô tả chính xác các yêu cầu mà sản phẩm phần mềm cần đáp ứng, và có giao diện thế nào. Tài liệu đặc tả là cơ sở để đội ngũ phát triển phần mềm xây dựng sản phẩm phần mềm. Khi nói đến thất bại trên đây là nói đến việc sản phẩm phần mềm không hoạt động đúng như đặc tả. Lỗi một khi được tiến hành có thể dẫn đến thất bại. Do đó, lỗi về bỏ quên được coi là tương ứng với các lỗi khi xây dựng đặc tả.

Kiểm chứng và thẩm định: Kiểm chứng (verification) và thẩm định (validation) hay được dùng lẫn lộn, nhưng thực ra chúng có ý nghĩa khác nhau. Kiểm chứng là quá trình để đảm bảo rằng một sản phẩm phần mềm thỏa mãn đặc tả của nó. Còn thẩm định là quá trình để đảm bảo rằng sản phẩm đáp ứng được yêu cầu của người dùng (khách hàng). Trong thực tế, chúng ta cần thực hiện kiểm chứng trước khi thực hiện việc thẩm định sản phẩm phần mềm. Vì vậy, chúng ta có thuật ngữ V&V (Verification & Validation). Lý do của việc này là chúng ta cần đảm bảo sản phẩm đúng với đặc tả trước. Nếu thực hiện việc thẩm định trước, một khi phát hiện ra lỗi, chúng ta không thể xác định được lỗi này do đặc tả sai hay do lập trình sai so với đặc tả.

Chất lượng và độ tin cậy của phần mềm: Theo từ điển, chất lượng của một sản phẩm được thể hiện bằng các đặc trưng phù hợp với đặc tả của nó. Theo cách hiểu này, chất lượng của một sản phẩm phần mềm là sự đáp ứng các yêu cầu về chức năng (tức là các hàm cần được tính toán), sự hoàn thiện và các chuẩn đã được đặc tả, cùng các đặc trưng mong chờ từ mọi sản phẩm phần mềm chuyên nghiệp. Chất lượng

phần mềm đặc trưng cho “độ tốt, độ tuyệt hảo” của phần mềm, và gồm có các yếu tố về chất lượng như: tính đúng đắn (hành vi đúng như đặc tả), tính hiệu quả (tiết kiệm thời gian và tiền bạc), độ tin cậy, tính khả kiểm thử (kiểm thử được và dễ), dễ học, dễ sử dụng, dễ bảo trì ... Như vậy, độ tin cậy chỉ là một yếu tố để đánh giá chất lượng phần mềm. Người kiểm thử hay nhầm lẫn độ tin cậy với chất lượng. Khi kiểm thử đạt tới mức phần mềm chạy ổn định, có thể phụ thuộc vào nó, người kiểm thử thường cho rằng phần mềm đã đạt chất lượng cao. Các yếu tố về mặt chất lượng mà liên quan trực tiếp đến việc phát triển phần mềm được gọi là các tiêu chuẩn chất lượng như tính có cấu trúc, tính đơn thể, tính khả kiểm thử, ...

Độ tin cậy của phần mềm là xác suất để phần mềm chạy không có thất bại trong một khoảng thời gian nhất định. Nó được xem là một yếu tố quan trọng của chất lượng phần mềm. Ngoài ra, thời gian trung bình cho việc khắc phục một sự cố cũng là một thông số quan trọng trong việc đánh giá độ tin cậy của sản phẩm phần mềm

Kiểm thử: Rõ ràng việc kiểm thử liên quan đến các khái niệm trên: lỗi, sai, thất bại và sự cố. Có hai mục đích chính của một phép thử: tìm thất bại hoặc chứng tỏ việc tiến hành của phần mềm là đúng đắn.

Vai trò của kiểm thử phần mềm: Kiểm thử phần mềm đóng vai trò quan trọng trong việc đánh giá và thu được chất lượng cao của sản phẩm phần mềm trong quá trình phát triển. Thông qua chu trình “*kiểm thử - tìm lỗi - sửa lỗi*”, ta hy vọng chất lượng của sản phẩm phần mềm sẽ được cải tiến. Mặt khác, thông qua việc tiến hành kiểm thử mức hệ thống trước khi cho lưu hành sản phẩm, ta biết được sản phẩm của ta tốt ở mức nào. Vì thế, nhiều tác giả đã mô tả việc kiểm thử phần mềm là một quy trình kiểm chứng để đánh giá và tăng cường chất lượng của sản phẩm phần mềm. Quy trình này gồm hai công việc chính là phân tích tĩnh và phân tích động.

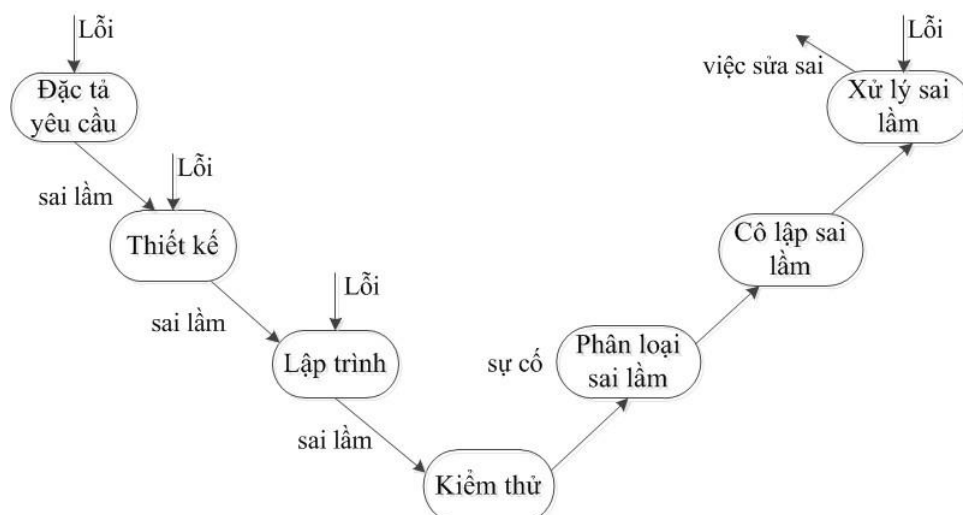
Phân tích tĩnh: Việc phân tích tĩnh được tiến hành dựa trên việc khảo sát các tài liệu được xây dựng trong quá trình phát triển sản phẩm như tài liệu đặc tả nhu cầu người dùng, mô hình phần mềm, hồ sơ thiết kế và mã nguồn phần mềm. Các phương pháp phân tích tĩnh truyền thống bao gồm việc khảo sát đặc tả và mã nguồn cùng các tài liệu thiết kế. Các kỹ thuật khảo sát này sẽ được giới thiệu trong chương 4. Người ta cũng có thể dùng các kỹ thuật phân tích hình thức như kiểm chứng mô hình (model checking) và chứng minh định lý (theorem proving) để chứng minh tính

đúng đắn của thiết kế và mã nguồn. Các kỹ thuật này tương đối phức tạp và nằm ngoài khuôn khổ của cuốn giáo trình này. Công việc này không dừng đến việc thực thi chương trình mà chỉ duyệt, lý giải về tất cả các hành vi có thể của chương trình khi được thực thi. Tối ưu hóa các chương trình dịch là các ví dụ về phân tích tĩnh.

Phân tích động: Phân tích động liên quan đến việc thực thi chương trình để phát hiện những thất bại có thể có của chương trình, hoặc quan sát các tính chất nào đó về hành vi và hiệu quả (performance). Vì gần như không thể thực thi chương trình trên tất cả các dữ liệu đầu vào có thể, ta chỉ có thể chọn một tập con các dữ liệu đầu vào để thực thi, gọi là các “ca kiểm thử”. Chọn như thế nào để được các bộ dữ liệu đầu vào hiệu quả (tức là các bộ dữ liệu có xác suất phát hiện thất bại (nếu có) cao hơn là công việc cần suy nghĩ và là nội dung chính của các giáo trình này

Bằng việc phân tích tĩnh và động, người kiểm thử muốn phát hiện nhiều lỗi nhất có thể được để chúng có thể được sửa ở giai đoạn sớm nhất trong quá trình phát triển phần mềm. Phân tích tĩnh và động là hai kỹ thuật bổ sung cho nhau và cần được làm lặp đi lặp lại nhiều trong quá trình kiểm thử.

Ca kiểm thử: Mỗi ca kiểm thử có một tên và được liên kết với một hành vi của chương trình. Ca kiểm thử gồm một tập các dữ liệu đầu vào và một xâu các giá trị đầu ra mong đợi đối với phần mềm



Hình 1.1: Một vòng đời của việc kiểm thử.

Hình 1.1 mô tả vòng đời của việc kiểm thử ứng với mô hình thác nước. Lưu ý rằng trong giai đoạn phát triển phần mềm, lỗi có thể được đưa vào tại các giai đoạn

đặc tả yêu cầu, thiết kế và lập trình. Các lỗi này có thể tạo ra những sai lan truyền sang các phần còn lại của quá trình phát triển. Một nhà kiểm thử lỗi lạc đã tóm tắt vòng đời này như sau: Ba giai đoạn đầu là “đưa lỗi vào”, giai đoạn kiểm thử là để tìm lỗi, và ba giai đoạn cuối là “khử lỗi đi” [Pos90]. Bước sửa sai là cơ hội mới cho việc đưa vào lỗi (và các sai mới). Vì vậy, việc sửa sai này có thể làm cho phần mềm từ đúng trở thành sai. Trong trường hợp này, việc sửa sai là không đầy đủ. Kiểm thử hồi quy (sẽ được giới thiệu trong chương 11) là giải pháp tốt để giải quyết vấn đề này.

Các thuật ngữ trên đây cho thấy các ca kiểm thử chiếm vị trí trung tâm trong việc kiểm thử dựa trên phân tích động. Quá trình kiểm thử dựa trên phân tích động được chia thành các bước sau: lập kế hoạch kiểm thử, phát triển ca kiểm thử, chạy các ca kiểm thử và đánh giá kết quả kiểm thử. Tiêu điểm của cuốn giáo trình này là việc xác định tập hữu ích các ca kiểm thử, tức là các ca kiểm thử giúp ta cải tiến tốt hơn chất lượng của sản phẩm

1.2. Ca kiểm thử phần mềm

Cốt lõi của kiểm thử phần mềm dựa trên phân tích động là việc xác định tập các ca kiểm thử sao cho chúng có khả năng phát hiện nhiều nhất các lỗi (có thể có) của hệ thống cần kiểm thử. Vậy cái gì cần đưa vào các ca kiểm thử? Rõ ràng thông tin đầu tiên là đầu vào. Đầu vào có hai kiểu: tiền điều kiện (pre-condition) - tức là điều kiện cần thỏa mãn trước khi tiến hành ca kiểm thử - và dữ liệu đầu vào thực sự được xác định bởi phương pháp kiểm thử. Thông tin tiếp theo cần đưa vào là đầu ra mong đợi. Cũng có hai loại đầu ra: hậu điều kiện (post-condition) và dữ liệu đầu ra thực sự. Phần đầu ra của ca kiểm thử thường hay bị bỏ quên vì nó là phần khó xác định. Giả sử ta cần kiểm thử phần mềm tìm đường đi tối ưu cho máy bay khi cho trước các ràng buộc về hành lang bay và dữ liệu về thời tiết trong ngày của chuyến bay. Đường đi tối ưu của máy bay thực sự là gì? Có nhiều câu trả lời cho câu hỏi này. Câu trả lời lý thuyết là giả thiết về sự tồn tại của một cây đũa thần (oracle) biết được tất cả các câu trả lời. Câu trả lời thực tế, được gọi là kiểm thử tham chiếu, là hệ thống được kiểm thử dưới sự giám sát của các chuyên gia về lĩnh vực ứng dụng của phần mềm, những người có thể phán xét xem liệu các dữ liệu đầu ra đối với việc tiến hành trên các dữ liệu đầu vào của ca kiểm thử có chấp nhận được hay không.

Hoạt động kiểm thử dẫn đến việc thiết lập các tiền điều kiện cần thiết, việc cung cấp các ca kiểm thử, quan sát dữ liệu đầu ra và so sánh chúng với các đầu ra mong đợi để xác định phát hiện các lỗi/khiếm khuyết (có thể có) của sản phẩm phần mềm.

Hình 1.2 mô tả các thông tin cơ bản trong một ca kiểm thử được phát triển đầy đủ, chủ yếu là để trợ giúp việc quản lý. Các ca kiểm thử cần phải định danh bằng tên/chỉ số và lý do tồn tại (chẳng hạn đặc tả nhu cầu tương ứng là một lý do). Cũng nên bổ sung thêm lịch sử tiến hành của một ca kiểm thử bao gồm cả việc chúng được thực hiện bởi ai và khi nào, kết quả của mỗi lần thực hiện ra sao, qua hay thất bại và được thực hiện trên phiên bản nào của phần mềm. Với các ca kiểm thử cho các hoạt động kiểm thử giao diện người dùng, ngoài thông tin về đầu vào, chúng ta cần bổ sung thêm các thông tin về trình tự nhập các đầu vào cho giao diện. Tóm lại, ta cần nhận thức rằng ca kiểm thử ít nhất cũng quan trọng như mã nguồn. Các ca kiểm thử cần được phát triển, kiểm tra, sử dụng, quản lý và lưu trữ một cách khoa học.

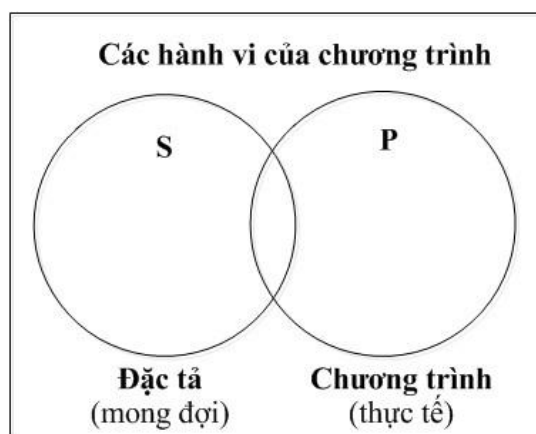
Chỉ số của ca kiểm thử			
Mục đích			
Tiền điều kiện			
Đầu vào			
Đầu ra mong đợi			
Hậu điều kiện			
Lịch sử thực hiện ca kiểm thử:			
Ngày	Kết quả thực tế	Phiên bản	Kiểm thử viên

Hình 1.2: Thông tin về một ca kiểm thử tiêu biểu

1.3. Mô tả bài toán kiểm thử qua biểu đồ Venn

Kiểm thử chủ yếu liên quan tới hành vi của chương trình nơi mà hành vi phản ánh quan điểm về cấu trúc phổ dụng đối với các nhà phát triển hệ thống hoặc phần mềm. Sự khác biệt là quan điểm cấu trúc tập trung vào “là cái gì”, còn quan điểm hành vi lại tập trung vào “làm gì”. Một trong những nguyên nhân gây khó cho người kiểm thử là các tài liệu cơ sở thường được viết bởi và viết cho người phát triển và vì thế chúng thiên về thông tin cấu trúc và coi nhẹ thông tin về hành vi của chương trình cần kiểm thử. Trong mục này, chúng ta phát triển một biểu đồ Venn đơn giản nhằm làm sáng tỏ vài điều về kiểm thử.

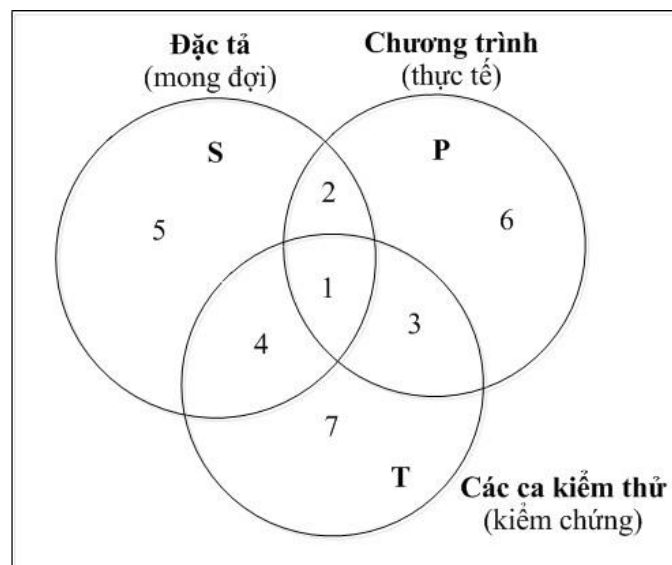
Xét một vũ trụ của hành vi chương trình cần kiểm thử, lưu ý rằng chúng ta đang quan tâm đến bản chất của việc kiểm thử là xác định tập các ca kiểm thử hữu ích. Cho trước một chương trình cùng đặc tả của nó. Gọi S là tập các hành vi được đặc tả và P là tập các hành vi của chương trình. Hình 1.3 mô tả mối quan hệ giữa vũ trụ các hành vi được lập trình và hành vi được đặc tả. Trong tất cả các hành vi có thể của chương trình, những hành vi được đặc tả nằm trong vòng tròn với nhãn S , còn những hành vi được lập trình là ở trong vòng tròn với nhãn P . Từ biểu đồ này, ta thấy rõ các bài toán mà người kiểm thử cần đối mặt là gì. Nếu có hành vi được đặc tả nhưng không được lập trình thì theo thuật ngữ trước đây, đây là những sai về bỏ quên. Tương tự, nếu có những hành vi được lập trình nhưng không được đặc tả, thì điều đó tương ứng với những sai về nhiệm vụ, và tương ứng với những lỗi xuất hiện sau khi đặc tả đã hoàn thành. Tương giao giữa S và P là phần đúng đắn, gồm có các hành vi vừa được đặc tả vừa được cài đặt. Chú ý rằng tính đúng đắn chỉ có nghĩa đối với đặc tả và cài đặt và là khái niệm mang tính tương đối.



Hình 1.3: Các hành vi được cài đặt và được đặc tả.

Vòng tròn mới (vòng tròn T) trong hình 1.4 là cho các ca kiểm thử. Lưu ý rằng tập các hành vi của chương trình nằm trọn trong vũ trụ chuyên đề mà ta đang xét. Vì một ca kiểm thử cũng xác định một hành vi (xin các nhà toán học sẽ bỏ qua cho việc dùng thuật ngữ quá tải này). Xét mối quan hệ giữa S , P và T . Có thể có các hành vi được đặc tả mà không được kiểm thử (các miền 2 và 5), các hành vi được đặc tả và được kiểm thử (các miền 1 và 4), và các ca kiểm thử tương ứng với các hành vi không được đặc tả (các miền 3 và 7).

Tương tự, có thể có các hành vi được lập trình mà không được kiểm thử (các miền 2 và 6), các hành vi được lập trình và được kiểm thử (các miền 1 và 3), và các ca kiểm thử tương ứng với các hành vi không được lập trình (các miền 4 và 7). Việc xem xét tất cả các miền này là hết sức quan trọng. Nếu có các hành vi được đặc tả mà không có các ca kiểm thử tương ứng, việc kiểm thử là chưa đầy đủ. Nếu có các ca kiểm thử tương ứng với các hành vi không được đặc tả, có thể có hai khả năng: hoặc đặc tả còn thiếu hoặc ca kiểm thử không đảm bảo. Theo kinh nghiệm, một người kiểm thử giỏi sẽ thường cho các ca kiểm thử thuộc loại đầu, và đây chính là lý do người kiểm thử cần tham gia vào giai đoạn khảo sát đặc tả và thiết kế.



Hình 1.4: Các hành vi được cài đặt, được đặc tả và được kiểm thử.

Ta có thể thấy việc kiểm thử như là công việc của một nghệ nhân: người kiểm thử có thể làm gì để làm cho miền tương giao (phần giao) của các tập (miền 1) là lớn nhất có thể? Làm thế nào để xác định các ca kiểm thử trong tập *T*? Câu trả lời là các ca kiểm thử cần được xác định bởi một phương pháp kiểm thử phù hợp.

1.4. Phân loại lỗi

Các định nghĩa của ta về lỗi và sai xoay quanh sự phân biệt giữa quy trình và sản phẩm: quy trình nói ta làm điều gì đó thế nào, còn sản phẩm là kết quả cuối cùng của quy trình. Kiểm thử phần mềm và đảm bảo chất lượng phần mềm (Software Quality Assurance - SQA) gặp nhau ở điểm là SQA cố gắng cải tiến chất lượng sản phẩm bằng việc cải tiến quy trình. Theo nghĩa này thì kiểm thử là định hướng sản phẩm. SQA quan tâm nhiều hơn đến việc giảm thiểu lỗi trong quá trình phát triển,

còn kiểm thử quan tâm chủ yếu đến phát hiện sai trong sản phẩm. Cả hai nguyên lý này đều sử dụng định nghĩa về các loại sai. Các sai được phân loại theo vài cách: giai đoạn phát triển khi cái sai tương ứng xuất hiện, các hậu quả của các thất bại tương ứng, độ khó cho việc giải quyết, độ rủi ro của việc không giải quyết được, vân vân. Một cách phân loại được ưa thích là dựa trên việc xuất hiện bất thường: chỉ một lần, thỉnh thoảng, xuất hiện lại hoặc lặp đi lặp lại nhiều lần. Hình 1.5 minh họa một cách phân loại sai dựa trên độ nghiêm trọng của hậu quả.

1 Nhẹ	Lỗi chính tả
2 Vừa	Hiểu lầm hoặc thừa thông tin
3 Khó chịu	Tên bị thiếu, cụt chữ hoặc hóa đơn có giá trị 0.0 đồng
4 Bực mình	Vài giao dịch không được xử lý
5 Nghiêm trọng	Mất giao dịch
6 Rất nghiêm trọng	Xử lý giao dịch sai
7 Cực kỳ nghiêm trọng	Lỗi rất nghiêm trọng xảy ra thường xuyên
8 Quá quắt	Hủy hoại cơ sở dữ liệu
9 Thảm họa	Hệ thống bị tắt
10 Dịch họa	Thảm họa lây lan

Hình 1.5: Phân loại sai bằng độ nghiêm trọng.

Để xử lý các loại sai, hãy tham khảo [IEE93] về việc phân loại chuẩn cho các bất thường của phần mềm. Chuẩn IEEE định nghĩa quy trình giải quyết bất thường một cách chi tiết gồm bốn giai đoạn: nhận biết, khảo sát, hành động và bố trí lại. Một số bất thường phổ biến được mô tả trong các bảng từ Bảng 1.1 đến Bảng 1.5. Hầu hết các bất thường này đều đã được đề cập trong chuẩn IEEE. Ngoài ra, chúng tôi cũng bổ sung thêm một số bất thường khác.

Bảng 1.1: Các sai lầm về đầu vào/đầu ra

Loại	Các trường hợp
dữ liệu đầu vào	dữ liệu đầu vào đúng nhưng không được chấp nhận
	dữ liệu đầu vào sai nhưng được chấp nhận
	mô tả sai hoặc thiếu
	tham số sai hoặc thiếu
dữ liệu đầu ra	khuôn dạng sai
	kết quả sai
	kết quả đúng tại thời gian sai (quá sớm hoặc quá muộn)
	kết quả không đầy đủ hoặc thiếu
	kết quả giả tạo
	văn phạm/chính tả

	các trường hợp khác
--	---------------------

Bảng 1.2: Các sai lầm về logic

trường hợp
lập thừa trường hợp
điều kiện cực đoan bị bỏ qua
thể hiện sai
thiếu điều kiện
điều kiện ngoại lai
kiểm thử sai biến
việc lập của chu trình không đúng
phép toán sai (chẳng hạn dùng $<$ cho $\leq$)

Bảng 1.3: Các sai lầm về tính toán

thuật toán sai
thiếu tính toán
toán hạng sai
sai về dấu ngoặc
chưa đủ độ chính xác (làm tròn hoặc cắt đuôi)
hàm đi kèm sai

Bảng 1.4: Các sai lầm về giao diện

xử lý gián đoạn sai (trong các hệ thống nhúng)
thời gian vào ra (time out)
gọi sai thủ tục
gọi đến một thủ tục không tồn tại
tham số không sánh cặp được (mismatch) (chẳng hạn về kiểu và số)
kiểu không tương thích
bao hàm thừa

Bảng 1.5: Các sai lầm về dữ liệu

khởi tạo sai
lưu trữ và truy cập sai
giá trị chỉ số và cờ báo sai
gói và mở sai
sử dụng sai biến
tham chiếu sai dữ liệu
đơn vị hoặc thang chia sai
chiều của dữ liệu sai
chỉ số sai
sai về kiểu
sai về phạm vi
dữ liệu cảm biến vượt ra ngoài miền cho phép
lỗi thừa, thiếu một số với biên
dữ liệu không tương thích

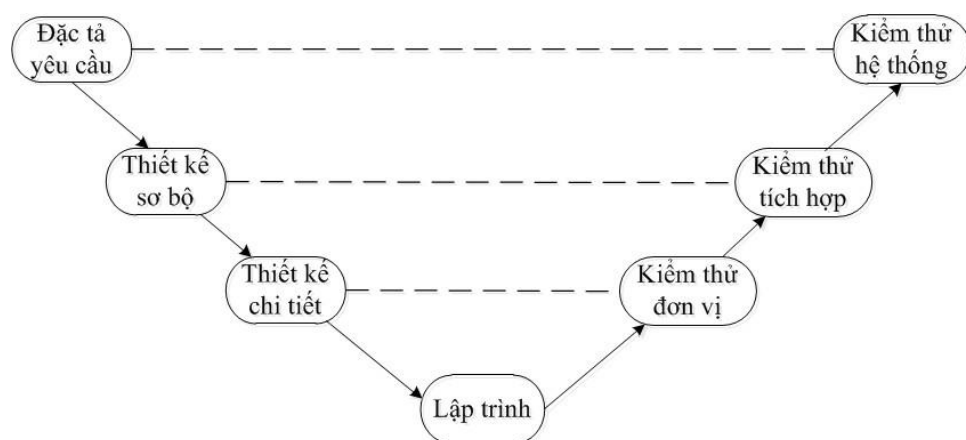
1.5. Các mức kiểm thử

Một trong các khái niệm then chốt của việc kiểm thử là các mức của việc kiểm thử. Các mức của việc kiểm thử phản ánh mức độ trừu tượng được thấy trong mô hình thác nước của vòng đời của việc phát triển phần mềm.

Dù có một số nhược điểm, mô hình này vẫn rất hữu ích cho việc kiểm thử, là phương tiện để xác định các mức kiểm thử khác nhau và làm sáng tỏ mục đích của mỗi mức. Một dạng của mô hình thác nước được trình bày trong hình 1.6. Dạng này nhấn mạnh sự tương ứng của việc kiểm thử với các mức thiết kế. Lưu ý rằng theo các thuật ngữ của việc kiểm thử hàm, ba mức của định nghĩa (đặc tả, thiết kế sơ bộ và thiết kế chi tiết) tương ứng trực tiếp với ba mức của việc kiểm thử là kiểm thử đơn vị, kiểm thử tích hợp và kiểm thử hệ thống. Các mức của kiểm thử cũng làm nảy sinh vấn đề về thứ tự kiểm thử: dưới lên, trên xuống hoặc các khả năng khác

Các mức kiểm thử có thể được mô tả sơ bộ như sau:

- Kiểm thử đơn vị: Kiểm thử đơn vị là việc kiểm thử các đơn vị chương trình một cách cô lập. Thế nào là một đơn vị chương trình? Câu trả lời phụ thuộc vào ngữ cảnh công việc. Một đơn vị chương trình là một đoạn mã nguồn như hàm hoặc phương thức của một lớp, có thể được gọi từ ngoài, và cũng có thể gọi đến các đơn vị chương trình khác. Đơn vị cũng còn được coi là một đơn thể để kết hợp. Đơn vị chương trình cần được kiểm thử riêng biệt để phát hiện lỗi trong nội tại và khắc phục trước khi được tích hợp với các đơn vị khác.



Hình 1.6: Các mức trừu tượng và mức kiểm thử trong mô hình thác nước.

- Kiểm thử tích hợp: Mức kế tiếp với kiểm thử đơn vị là kiểm thử tích hợp. Sau khi các đơn vị chương trình đã cấu thành hệ thống đã được kiểm thử,

chúng cần được kết nối với nhau để tạo thành hệ thống đầy đủ và có thể làm việc. Công việc này không hề đơn giản và có thể có những lỗi về giao diện giữa các đơn vị, và cần phải kiểm thử để phát hiện những lỗi này. Công đoạn này gồm hai giai đoạn: giai đoạn kiểm thử tích hợp và giai đoạn kiểm thử hệ thống. Kiểm thử tích hợp nhằm đảm bảo hệ thống làm việc ổn định trong môi trường thí nghiệm để sẵn sàng cho việc đưa vào môi trường thực sự bằng cách đặt các đơn vị với nhau theo phương pháp tăng dần.

- Kiểm thử hệ thống: Kiểm thử mức này được áp dụng khi đã có một hệ thống đầy đủ sau khi tất cả các thành phần đã được tích hợp. Mục đích của kiểm thử hệ thống là để đảm bảo rằng việc cài đặt tuân thủ đầy đủ các yêu cầu được đặc tả của người dùng. Công việc này tốn nhiều công sức, vì có nhiều khía cạnh về yêu cầu người dùng cần được kiểm thử.
- Kiểm thử chấp nhận: Khi nhóm kiểm thử hệ thống đã thỏa mãn với một sản phẩm, sản phẩm đó đã sẵn sàng để đưa vào sử dụng. Khi đó hệ thống cần trải qua giai đoạn kiểm thử chấp nhận. Kiểm thử chấp nhận được thực thi bởi chính các khách hàng nhằm đảm bảo rằng sản phẩm phần mềm làm việc đúng như họ mong đợi. Có hai loại kiểm thử chấp nhận: kiểm thử chấp nhận người dùng, được tiến hành bởi người dùng, và kiểm thử chấp nhận doanh nghiệp, được tiến hành bởi nhà sản xuất ra sản phẩm phần mềm.

1.6 Các nguyên tắc kiểm thử phần mềm

Kiểm thử phần mềm là quá trình vận hành một chương trình nhằm tìm ra lỗi của nó. Để phần mềm hoạt động trơn tru, nó cần phải sạch lỗi. Và nếu kiểm thử phần mềm được thực hiện thành công, việc này sẽ khiến các lỗi không còn xuất hiện. Tuy nhiên, để tiết kiệm thời gian và công sức truy tìm các lỗi, có **7 nguyên tắc kiểm thử** quan trọng mà chúng ta cần tuân theo.

Nguyên tắc 1. Kiểm thử phần mềm chứng minh sự hiện diện của lỗi

Bằng việc kiểm thử, chúng ta có thể làm giảm lượng bugs khi áp dụng nhiều phương pháp kiểm thử lên phần mềm. Tuy nhiên khi được đưa lên môi trường thật, người dùng cuối hoàn toàn có thể thấy nhiều lỗi khác không tìm thấy trong quá trình kiểm thử. Kiểm thử chứng minh được sản phẩm có lỗi nhưng không thể chứng minh rằng sản phẩm không còn lỗi. Điều này có nghĩa là, sẽ luôn có lỗi không được phát hiện trong phần

mềm, ngay cả khi không tìm thấy lỗi, cũng không đồng nghĩa rằng phần mềm đúng hoàn toàn.

Nguyên tắc 2. Kiểm thử toàn bộ là không khả thi

Đúng vậy, rất khó để kiểm tra toàn bộ các module cũng như các tính năng, kết hợp với đầu vào và đầu ra trong suốt quá trình kiểm tra. Các sản phẩm phần mềm hiện nay cực kỳ đa dạng và phức tạp, được phát triển trên nhiều nền tảng, thêm vào đó, ngày càng có nhiều công nghệ mới, khả năng kết nối dữ liệu lớn... khiến việc kiểm thử toàn bộ gần như là không thể. Thay vì cố gắng kiểm thử toàn bộ, hãy xác định mức độ quan trọng và độ ưu tiên của các module để kiểm thử những phần cần thiết hoặc gặp nhiều nguy cơ hơn.

Nguyên tắc 3. Kiểm thử càng sớm càng tốt

Nguyên tắc kiểm thử sớm có nghĩa là việc kiểm thử cần được thực hiện càng sớm càng tốt trong vòng đời phát triển phần mềm. Vậy ở bước nào thì được coi là sớm? Nguyên tắc này cho thấy ta cần phát hiện bug ngay từ các bước đầu tiên như nghiên cứu yêu cầu (requirement) hay design. Phát hiện lỗi càng muộn, chi phí bỏ ra để xử lý càng cao. Vì vậy, việc thay đổi yêu cầu không đúng từ sớm sẽ khiến tốn ít chi phí để thay đổi tính năng hơn.

Nguyên tắc 4. Lỗi thường được phân bố tập trung

Nguyên lý về phân bố lỗi chỉ ra rằng, chỉ một số ít module chứa phần lớn số lỗi phát hiện được. Những module này thường là những thành phần, chức năng chính của hệ thống. Điều này cũng đúng theo nguyên lý Pareto: 80 – 20: 80% số lỗi tìm thấy ở chỉ 20% module. Bằng kinh nghiệm, các QA/ Tester có thể xác định được những module có tính rủi ro và nhiều lỗi như vậy, giúp tập trung tìm kiếm lỗi nhanh và hiệu quả hơn. Tuy nhiên, cách tiếp cận này cũng ẩn chứa vấn đề: nếu thực hiện kiểm thử tương tự lặp đi lặp lại, dễ thấy rằng những test case cũ sẽ khó tìm thêm được bug mới.

Nguyên tắc 5. Nghịch lý thuốc trừ sâu

Trong trồng trọt, nếu người nông dân sử dụng lặp lại một liều trừ sâu, các loại sâu bệnh sẽ dần dần thích nghi và trở nên “nhờn” với loại thuốc trừ sâu đó. Tương tự với kiểm thử phần mềm, khi lặp đi lặp lại một test case, thì xác suất tìm được lỗi là rất thấp. Nguyên nhân là hệ thống hoàn thiện hơn, lỗi tìm thấy đã được sửa theo test case cũ. Để

xử lý hiệu ứng “thuốc trừ sâu” này, test case cần được thường xuyên xem lại và chỉnh sửa, thêm nhiều test case mới để tìm lỗi mới (regression test).

Thêm vào đó, QA/ Tester cũng không nên phụ thuộc quá nhiều vào các kỹ thuật test sẵn có. Bạn cần liên tục cải tiến các phương pháp có sẵn để làm việc kiểm thử hiệu quả hơn.

Nguyên tắc 6. Kiểm thử phụ thuộc vào ngữ cảnh

Kiểm thử phụ thuộc vào ngữ cảnh, nói một cách đơn giản, là việc kiểm thử một trang thương mại điện tử sẽ phải khác cách test một ứng dụng đọc tin tức. Tất cả các phần mềm đều được phát triển theo cách khác nhau. Và việc áp dụng chung một “cách giải” là sai lầm. Bạn cần sử dụng cách tiếp cận khác nhau, phương thức, kỹ thuật test khác nhau, loại test phụ thuộc vào loại phần mềm/ ứng dụng/ website.

Nguyên tắc 7. Quan niệm sai lầm về việc “hết lỗi”

Một phần mềm sạch bug 99% vẫn có thể không sử dụng được. Đây là trường hợp phần mềm được kiểm thử bằng một requirement sai. Kiểm thử không chỉ để tìm ra lỗi, mà còn để kiểm tra xem phần mềm có đáp ứng được đúng nhu cầu hay không. Chính vì vậy, việc Không còn lỗi hay Hết lỗi là quan niệm sai lầm.

Quan điểm: “Nguyên tắc kiểm thử chỉ là để tham khảo, không có tính ứng dụng thực tế”?

Đây là quan điểm cực kỳ sai lầm. Các nguyên tắc kiểm thử giúp tạo ra một chiến lược kiểm thử rõ ràng và tạo ra những trường hợp kiểm thử sát sao, để bắt được bug. Những tester dày dặn kinh nghiệm áp dụng những nguyên tắc kiểm thử nhuần nhuyễn đến độ họ không nghĩ rằng họ đang áp dụng chúng. Vì vậy, việc nguyên tắc kiểm thử không có tính ứng dụng thực tế là sai lầm.

1.7 Đảm bảo chất lượng phần mềm

Theo Daniel Galin, khái niệm đảm bảo chất lượng phần mềm được xác định như sau :

Đảm bảo chất lượng phần mềm là một tập các hoạt động đã được lập kế hoạch và có hệ thống, cần thiết để cung cấp đầy đủ sự tin cậy vào quy trình phát triển phần mềm hay quy trình bảo trì phần mềm của sản phẩm hệ thống phần mềm phù hợp với các yêu cầu chức năng kỹ thuật cũng như với các yêu cầu quản lý mà giữ cho lịch biểu và hoạt động trong phạm vi ngân sách.

1.7.1 Những mục tiêu đảm bảo chất lượng phần mềm

Phát triển phần mềm luôn đi đôi với bảo trì, vì vậy các hoạt động bảo đảm chất lượng phần mềm đều có mối liên quan chặt chẽ đến bảo trì. Những mục tiêu đảm bảo chất lượng phần mềm tương ứng với giai đoạn phát triển và bảo trì được xác định cụ thể như sau :

- Phát triển phần mềm (hướng tiến trình)

1. Đảm bảo một mức độ chấp nhận được rằng phần mềm sẽ thực hiện được các yêu cầu chức năng.
2. Đảm bảo một mức độ chấp nhận được rằng phần mềm sẽ đáp ứng được các yêu cầu về lịch biểu và ngân sách
3. Thiết lập và quản lý các hoạt động để cải thiện và nâng cao hiệu quả của phát triển phần mềm và các hoạt động SQA.

- Bảo trì phần mềm (hướng sản phẩm)

1. Đảm bảo một mức độ chấp nhận được rằng các hoạt động bảo trì phần mềm sẽ đáp ứng được các yêu cầu chức năng.
2. Đảm bảo một mức độ chấp nhận được rằng các hoạt động bảo trì phần mềm sẽ đáp ứng được các yêu cầu về lịch biểu và ngân sách
3. Thiết lập và quản lý các hoạt động để cải thiện và nâng cao hiệu quả của bảo trì phần mềm.

1.7.2 Phân loại yêu cầu phần mềm ứng với các yếu tố chất lượng phần mềm

Đã có nhiều tác giả nghiên cứu về các yếu tố chất lượng phần mềm từ các yêu cầu của nó. Theo thời gian có thể quan niệm về việc đảm bảo chất lượng phần mềm có phần thay đổi, tuy nhiên mô hình các yếu tố đảm bảo chất lượng phần mềm của McCall ra đời vào những năm 70 của thế kỷ trước vẫn còn được nhiều người nhắc đến như là cơ sở tham chiếu các yêu cầu phần mềm. Sau McCall cũng có một số mô hình được quan tâm như mô hình do Evans, Marciniak do hay mô hình của Deutsch và Willis, tuy nhiên những mô hình này chỉ bổ sung hay sửa đổi một vài yếu tố chất lượng. Theo McCall, các yếu tố chất lượng phần mềm được chia làm ba loại:

- Các yếu tố hoạt động của sản phẩm bao gồm tính chính xác, tin cậy, hiệu quả, tính toàn vẹn, sử dụng được

- Các yếu tố rà soát bao gồm tính bảo trì, linh hoạt, có thể test được

Các yếu tố chuyển giao bao gồm tính khả chuyển, có khả năng sử dụng lại, có khả năng giao tác.

Bài tập cuối bài

Bài 1.1. Các định nghĩa và thuật ngữ cơ bản về kiểm thử phần mềm?

Bài 1.2. Mô tả mỗi miền trong bảy miền trong hình 1.4?

Bài 1.3. Hãy vẽ biểu đồ Venn phản ánh khẳng định: ta đã không làm cái mà lẽ ra ta cần phải làm, và làm cái mà lẽ ra ta không được làm?

Bài 1.4. Phân loại lỗi?

Bài 1.5. Phân biệt các mức kiểm thử?

Bài 1.6. Trình bày các nguyên tắc kiểm thử?

Bài 1.7. Một trong các câu chuyện cũ về lĩnh vực phần mềm mô tả một nhân viên cấu hình viết một chương trình quản lý lương. Chương trình có chức năng kiểm tra số chứng minh thư của cán bộ và nhân viên trước khi đưa ra bản tính lương. Nếu có lúc người nhân viên này bị đuổi việc, chương trình sẽ tạo ra một mã độc gây hại cho cơ quan. Hãy bàn về tình trạng này theo các thuật ngữ trên đây về lỗi, sai, dạng thất bại và quyết định dạng kiểm thử nào là thích hợp.

Bài 1.8. Hãy so sánh hai cách tiếp cận kiểm thử hàm và kiểm thử cấu trúc?

Bài 1.9. Mục tiêu đảm bảo chất lượng phần mềm?

Bài 1.10. Phân loại yêu cầu phần mềm ứng với các yếu tố chất lượng phần mềm?